JAVA 데이터 처리

데이터란?

어떠한 현상이나 상태를 관찰을 한 후 그 결과로 나타나는 정성적 혹은 정량적인 실제 존재하는 값

정량적 데이터

수치로 측정 · 표현이 가능한 데이터

예 가격, 제품의 개수, 매출액

정성적 데이터

수치로 측정 · 표현이 불가능한 데이터

예 전화번호, 이름, 성별

데이터 출력

데이터를 콘솔 화면에 출력할 때 아래의 명령문을 이용해 데이터를 출력

System.out.print()	괄호 안의 내용을 출력
System.out.println()	괄호 안의 내용을 출력하고 한 행을 띄움
System.out.printf()	문자열을 서식문자를 이용하여 형식화된 내용으로 출력

데이터 출력

System.out.print()

```
package chapter02;

public class ConsolePrint {

    public static void main(String[] args) {

        System.out.print("안녕하세요."); // 괄호() 안의 데이터를 콘솔창에 출력
        System.out.print("저는 JAVA입니다."); //"안녕하세요" 문자열 옆에 문자열이 출력

    }

}
```

데이터 출력

System.out.println()

```
package chapter02;

public class ConsolePrintln {

    public static void main(String[] args) {

        System.out.println("안녕하세요."); // 문자열을 출력하고 한 줄을 띄어줌
        System.out.println("저는 JAVA입니다."); // “안녕하세요”가 출력되고 아래 줄에 출력

    }

}
```

데이터 출력

System.out.printf()

여러 서식문자를 이용하여 정해진 형식이 아닌 사용자가 원하는 형식의 문자열로 만들어 출력

서식문자	설명
%d	정수 형식으로 출력
%o	8진수 정수의 형식으로 출력
%x 또는 %X	16진수 정수의 형식으로 출력
%f	소수점 형식으로 출력
%c	문자 형식으로 출력
%s	문자열 형식으로 출력
%e 또는 %E	지수 표현식의 형식으로 출력

데이터 출력

System.out.printf()

```
package chapter02;

public class ConsolePrintf {
    public static void main(String[] args) {
        System.out.printf("안녕하세요. 저는 %s입니다.₩n", "JAVA"); // %s에 “JAVA”를 출력
        System.out.printf("저는 %d살 입니다. ₩n", 20); // %d에 20을 출력
        System.out.printf("정수 : %d, 실수 : %f ₩n", 10, 13.5);
        // %d에 10, %f에 13.5 출력
        System.out.printf("문자 : %c, 문자열 : %s ₩n", 'Z', "KING");
        // %c에 문자 Z, %s에 “KING” 문자열을 출력
    }
}
```

데이터 출력

System.out.printf()

```
System.out.printf("정수 : %d, 실수 : %f, 문자 : %c, 문자열 : %s", 10, 130.5, 'Z', "KING");
```

정수 : 10, 실수 : 130.500000, 문자 ; Z, 문자열 : KING

데이터 출력

이스케이프 시퀀스(Escape Sequence)

₩(\)로 시작해서 약속된 특정 의미로 해석되는 문자들

서식문자	설명
\n	다음 행으로 이동
\t	다음 탭으로 이동
\b	뒤로(왼쪽) 한 칸 이동
\r	줄의 맨 앞으로 이동
\\	\ 출력
\'	' 출력
\"	" 출력

데이터 출력

이스케이프 시퀀스(Escape Sequence)

```
package chapter02;

public class ConsolePrintlnEscSeq {

    public static void main(String[] args) {
        System.out.print("\n줄바꿈\n연습\n");
        System.out.print("\t텝키\t연습\n");
        System.out.print("이것은\r 앞으로 이동합니다.\n");
        System.out.print("글자가 \"강조\" 됩니다.\n");
        System.out.print("\\\\\\역슬래시 세개 출력");
    }
}
```

키보드를 이용한 데이터의 입력

사용자가 키보드를 통해 입력하는 데이터를 얻기 위해서는 java.util 패키지에 있는 Scanner 클래스를 이용

```
Scanner in = new Scanner(System.in);
```

키보드를 이용한 데이터의 입력

Scanner 클래스의 메소드

메소드	설명
nextBoolean()	사용자가 입력한 값을 boolean 타입의 데이터로 반환
nextByte()	사용자가 입력한 값을 byte 타입의 데이터로 반환
nextDouble()	사용자가 입력한 값을 double 타입의 데이터로 반환
nextFloat()	사용자가 입력한 값을 float 타입의 데이터로 반환
nextInt()	사용자가 입력한 값을 int 타입의 데이터로 반환
nextLine()	사용자가 입력한 값을 String 타입의 데이터로 반환
nextLong()	사용자가 입력한 값을 long 타입의 데이터로 반환
nextShort()	사용자가 입력한 값을 short 타입의 데이터로 반환

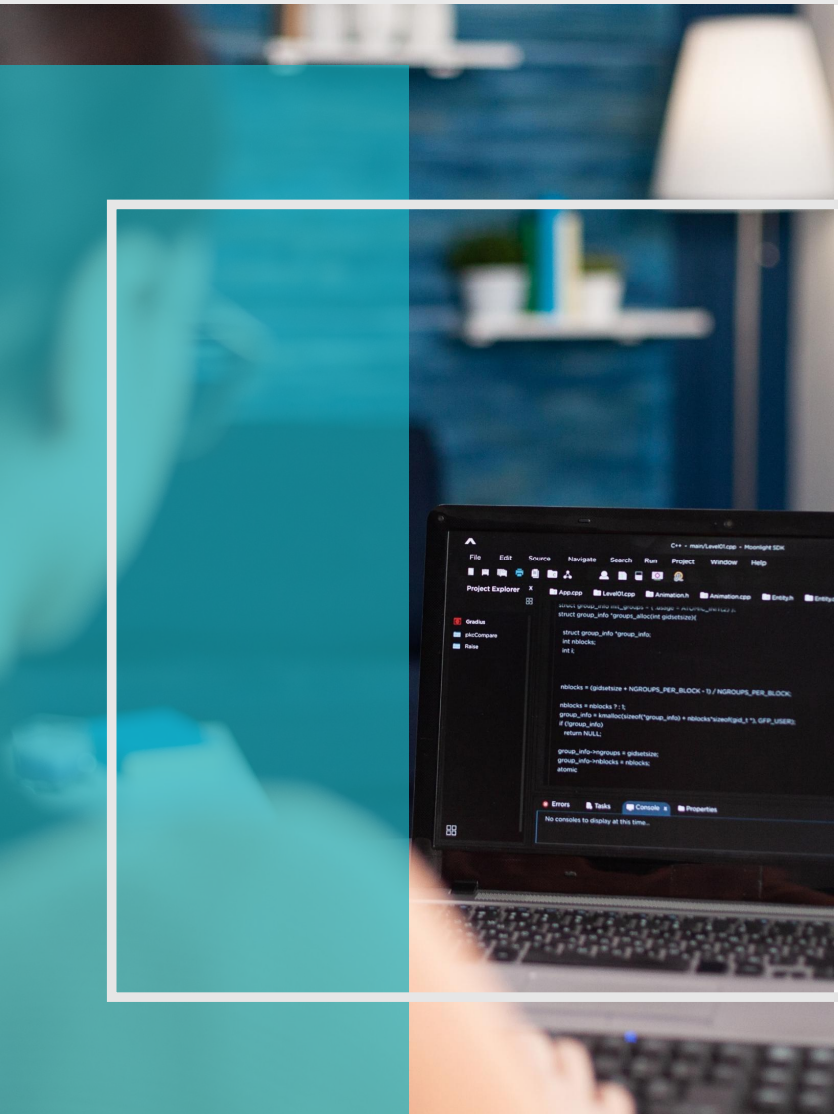
키보드를 이용한 데이터의 입력

```
package chapter02;

import java.util.Scanner;

public class ScannerTest {
	public static void main(String[] args) {
		Scanner in = new Scanner(System.in);
		System.out.print("이름을 입력하세요 >> ");
		String name = in.nextLine();
		System.out.println("안녕하세요. " + name + "님. 반갑습니다.");
		in.close();
	}
}
```

변수

COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
Java
Thomas Wu
C++

변수와 변수의 역할



변수

프로그램이 실행되면서
데이터들을 저장하고 데이터를
변경할 수 있도록 해주는 것

변수와 변수의 역할

변수는 데이터를 저장할 수 있는 메모리 공간을 가짐

저장하는 공간으로 찾아갈 수 있는 이름을 가짐

변수의 선언

변수의 선언 방법은 저장하고자 하는 데이터의 타입을 지정하고
식별할 수 있는 이름을 붙여주면 변수가 선언됨

```
int point ;
```

자료형 이름
이름

```
int num1;
double num2;
```

정수 데이터만
저장해야 함

정수타입의 변수 num1

실수 데이터만
저장해야 함

실수타입의 변수 num2

변수의 선언

```
int num1;
int num2;
```

다른 형태의 선언

```
int num1, int num2;
```

자료형 선언 없이 ,로 선언 가능

가능한 선언 형식

```
int num1, num2;
float num3, num4;
```

불가능한 선언 형식

```
int num1, float num3;
int num2, float num4;
```

변수에 데이터 대입하기

변수에 데이터의 대입은 변수 선언한 타입과 같은 데이터만 대입이 가능

정수 데이터만 저장해야 함

1000

정수타입의 변수 num

실수 데이터만 저장해야 함

3.14

실수타입의 변수 number

```
int num;
float number;
num = 1000;
number = 3.14f;
```

→

```
int num = 1000;
float number = 3.14f;
```

```
int num1;
int num2;
num1 = 1000;
num2 = 10000;
```

→

```
int num1 = 1000,
num2 = 10000;
```

변수에 데이터 대입하기

```
package chapter02;

public class Variable {
	public static void main(String[] args) {
		// 자료형 변수이름(식별을 위한 이름) = 데이터 ;
		int num = 1000;
		// 숫자 1000을 저장하기 위한 메모리 공간을 만들고, 저장공간의 주소를 가지는 이름 num을 만듦.
		int a;
		double b;
		int n1;
		int n2;
		// int n1, n2; 16번 17번 행의 선언과 동일합니다.
		int a1;
		float b1;
		int c1;
		float d1;
		// int a1, c1;
		// float b1, d1;
		// int a1, float b1; // 오류
		// int c1, float d1; // 오류
	}
}
```

변수의 이름(식별자) 명명 규칙

JAVA에서 식별자의 이름을 정할 때는 아래의 규칙에 맞게 정의함

1 대소문자가 구분되며 길이에 제한이 없음

- Point와 point는 서로 다른 것으로 인식됨

2 숫자로 시작해서는 안 됨

- top10은 허용하지만, 10top은 허용되지 않음

3 특수문자는 '_'와 '$'만을 허용함

- $dallar은 허용되지만 #Sarp는 허용되지 않음

4 키워드(예약어)를 사용해서는 안 됨

- true는 예약어라 사용할 수 없지만, True는 사용이 가능함

END

데이터 타입

COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
Java
Thomas Wu
C++

실제 메모리에 저장되는 단위인 bit

변수에 데이터를 저장할 때 메모리에는 0 또는 1의 값을 가지는 bit 단위로 저장

메모리에 데이터를 저장하는 것도 대입연산으로 처리되는데 이때 bit 단위로의 연산보다는
8개의 bit를 묶어 연산

byte	8개의 bit를 묶은 단위

JAVA에서는 데이터 자료형 타입의 저장 범위를 byte로 표현

bit 0 또는 1의 값만 가지는 단위

0	0	0	0	0	0	0	1

byte 8개의 bit를 가지는 단위

기본형 데이터 타입

기본형 타입은 참/거짓 데이터, 문자 데이터, 정수, 실수를 다룰 수 있는 자료형들이 있음

자료형	데이터	메모리 크기	저장 가능 범위
boolean	참, 거짓	1 byte	true, false
char	문자	2 byte	유니코드 값
byte	정수	1 byte	-2^{7}~$-2^{7}-1$ (-128~127)
short		2 byte	-2^{15}~$2^{15}-1$
int		4 byte	-2^{31}~$2^{31}-1$
long		8 byte	-2^{31}~$2^{31}-1$
float	실수	4 byte	1.4×10^{-45}~3.4×10^{38}
double		8 byte	4.9×10^{-324}~1.8×10^{308}

기본형 데이터 타입

참, 거짓 데이터를 저장하는 boolean

boolean타입은 논리 값(참/거짓)을 저장하는 자료형 타입

true

'참'의 값을 의미하는 상수 키워드

```
boolean check = true;
```

false

'거짓'의 값을 의미하는 상수 키워드

```
boolean power = false;
```

기본형 데이터 타입

참, 거짓 데이터를 저장하는 boolean

```
package chapter02;

public class VariableBooleanType {

    public static void main(String[] args) {
        // 참/거짓 저장 타입 : boolean
        boolean bool1 = true; // boolean 타입의 변수선언과 논리값 true를 초기화
        boolean bool2 = false; ; // boolean 타입의 변수선언과 논리값 false를 초기화

        System.out.println(bool1);
        System.out.println(bool2);
    }
}
```

기본형 데이터 타입

문자 데이터를 저장하는 char

char 타입은 문자 하나를 2바이트로 표현하는 유니코드 값으로 저장

유니코드	전 세계의 문자를 표현할 수 있는 코드의 집합 문자를 표현할 때는 작은따옴표 안에 문자 하나를 넣어 표현

```
char ch = 'A';
```

```
char c = 'a';
```

기본형 데이터 타입

문자 데이터를 저장하는 char

```
package chapter02;

public class VariableCharType {
	public static void main(String[] args) {
		// 문자 데이터 저장 타입 : char
		char ch1 = 'A'; // 문자 타입의 변수 선언과 변수에 데이터 'A' 값을 초기화
		char ch2 = '자'; // 문자 타입의 변수 선언과 변수에 데이터 '자'로 초기화
		char ch3 = 65; // Unicode 값 65를 직접 대입
		char ch4 = 51088; // Unicode 값 51088 값을 직접 대입
		System.out.println(ch1);
		System.out.println(ch2);
		System.out.println(ch3);
		System.out.println(ch4);
	}
}
```

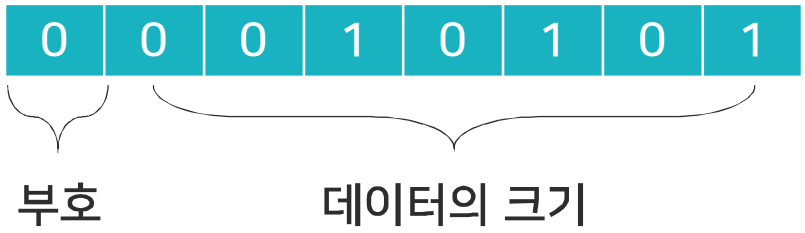
기본형 데이터 타입

정수의 표현

부호 비트	가장 왼쪽에 있는 부호를 표현하는 bit

부호 비트를 제외한 나머지 bit가 데이터 타입의 크기를 나타냄

이 bit의 사이즈가 데이터의 표현(저장) 하는 범위를 나타냄



부호 비트 0 → 양의 정수

0010101 → 21

결과: 양의 정수 → 21

기본형 데이터 타입

정수의 표현

정수를 저장하는 byte

-128~127까지의 값을 저장

보통 파일의 입출력 시 사용하는 타입으로 byte 타입의 배열로 처리

byte 변수이름 = 숫자 데이터

```
byte num = 100;
```

기본형 데이터 타입

정수의 표현

정수를 저장하는 short

-32,768 ~ 32,767까지의 값을 저장

short 변수이름 = 숫자 데이터

```
short num = 100;
```

기본형 데이터 타입

정수의 표현

정수를 저장하는 int

-2,147,483,648 (-2 ^ 31)에서 2,147,483,647 (2 ^31-1)까지의 값을 저장

JAVA의 정수 연산 시에는 int 타입 보다 작은 사이즈의 정수형 타입 변수들은
모두 int 타입으로 변경되어 연산

int 변수이름 = 숫자 데이터

```
int num = 100;
```

기본형 데이터 타입

정수의 표현

정수를 저장하는 long

-9,223,372,036,854,775,808 (-2 ^ 63)에서 9,223,372,036,854,775,807 (2 ^ 63-1) 까지의 값을 저장

long 변수이름 = 숫자 데이터

```
long num = 100;
```

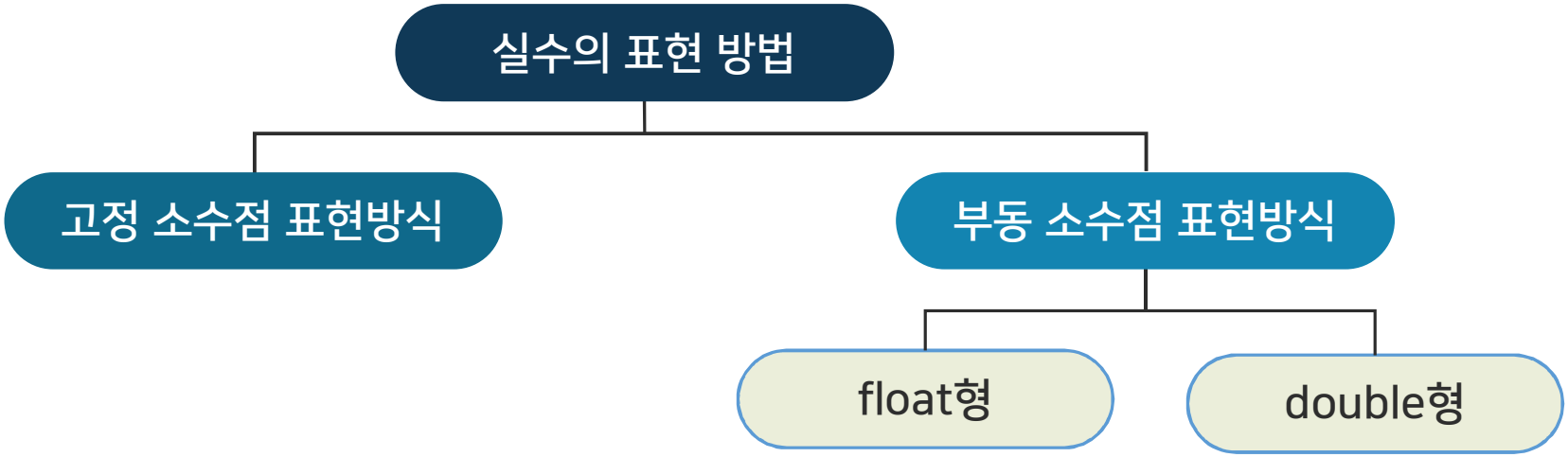
기본형 데이터 타입

정수의 표현

```
package chapter02;

public class VariableIntegerType {
    public static void main(String[] args) {
        // 정수형 데이터 저장 타입 : byte, short, int, long
        byte byteNum = 10; // 정수 타입의 변수 선언과 데이터 10을 저장
        short shortNum = 300; // 정수 타입의 변수 선언과 데이터 300을 저장
        int intNum = 100000; // 정수 타입의 변수 선언과 데이터 100000을 저장
        long longNum = 100000000000L; // 정수 타입의 변수 선언과 데이터 100000000000L을 저장
        System.out.println(byteNum);
        System.out.println(shortNum);
        System.out.println(intNum);
        System.out.println(longNum);
    }
}
```

기본형 데이터 타입

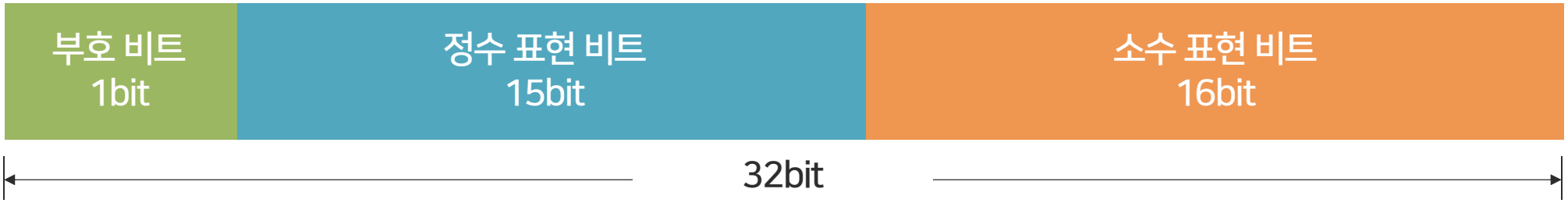
실수의 표현



기본형 데이터 타입

실수의 표현

고정 소수점 표현 방식



기본형 데이터 타입

실수의 표현

부동 소수점 표현 방식

float형	소수점 7자리까지 표현 가능

$$\pm(1.\text{가수부})\times 2^{\text{지수부}-127}$$

부호 비트 1bit	지수부 표현 비트 8bit	가수부 표현 비트 23bit

32bit

double형	소수점 15자리까지 표현 가능

부호 비트 1bit	지수부 표현 비트 11bit	가수부 표현 비트 52bit

64bit

기본형 데이터 타입

실수의 표현

실수를 저장하는 float

32bit 부동 소수점 형식으로 소수점 7자리까지 표현하고, 값은 무한대의 값을 저장

데이터의 저장 시에는 0.0f로 작성

정밀한 표현보다는 많은 값을 표현하는 형태의 표현이기 때문에 연산 시에 정확한 값을 얻지 못함

float 변수이름 = 숫자 데이터f

```
float num = 100.0f;
```

기본형 데이터 타입

실수의 표현

실수를 저장하는 double

64bit 부동 소수점 형식으로 소수점 15자리까지 표현하고, 값은 무한대의 값을 저장

실수의 표현에서는 double 값을 기본 타입으로 처리

기본 값을 표현할 때는 0.0d로 표현하지만 접미사 d는 생략 가능

```
double 변수이름 = 숫자 데이터d
```

```
double num = 100;
```

기본형 데이터 타입

실수의 표현

```
package chapter02;

public class VariableRealNumType {

    public static void main(String[] args) {

        // 실수 데이터 저장 타입 : float, double
        float num1 = 1.23456789f; // 데이터 1.2345679f을 저장
        double num2 = 1.23456789123456789; // 데이터 1.234567891234568을 저장

        System.out.println(num1);
        System.out.println(num2);
    }
}
```

참조형 데이터 타입

기본형 타입 8가지를 제외한 나머지 자료형을 참조형이라고 함

인스턴스의 메모리 주소를 저장하는데 사용

4바이트 사이즈의 저장 공간을 가짐

참조형 변수는 인스턴스의 메모리 주소를 저장하는 변수임

참조형 데이터 타입

```
package chapter02;

public class VariableRefferenceType {

    public static void main(String[] args) {

        String str = "JAVA Program"; //문자열 “JAVA Program”을 저장하고 있는 인스턴스의 주소 값을 저장

        System.out.println(str);

    }
}
```

변수의 초기화

변수의 초기화는 변수 선언 후 처음으로 값을 대입하는 것을 의미함

변수가 선언되는 위치에 따라 초기화 생략이 가능함

초기화를 생략하는 경우에는 다음 표와 같이 각 타입별로 기본값이 자동으로 초기화됨

자료형	기본 값
boolean	false
char	‘₩u0000’
byte	0
short	0
int	0

자료형	기본 값
long	0L
float	0.0f
double	0.0d
참조형	null

리터럴

프로그램에서 변수가 아닌 정수, 실수, 논리값, 문자, 문자열, null 과 같이 값 자체로 의미를 가지는 것

변수처럼 메모리의 저장공간에 값을 저장하고 사용되지만, 변수와 같이
식별할 수 있는 이름을 가지지 않음

리터럴의 접미사

리터럴도 메모리를 사용하기 때문에 리터럴이 저장되는 메모리 공간의 사이즈를 정의해 주어야 함

변수처럼 선언하는 과정이 없이 몇 가지 규칙을 통해 리터럴을 메모리에 저장

정수의 경우

기본적으로 int 타입의
저장 공간에 저장하고,
long 타입의 리터럴을 사용한다면
리터럴 뒤에 문자 L을 붙임

```
int num1 = 100;
long num2 = 10000000000L;
```

실수의 경우

기본적으로 double 타입의
저장 공간에 저장하고,
float 타입의 리터럴을 사용한다면
리터럴 뒤에 문자 F를 붙임

```
float num1 = 0.1f;
double num2 = 0.1;
```

데이터 자료형의 변환

COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
Java
Thomas Wu
C++

데이터 자료형의 변환

데이터 표현 방법이 제각각이기 때문에 연산을 하는 경우 표현 방법을 일치시켜야 정상적인 연산이 가능함

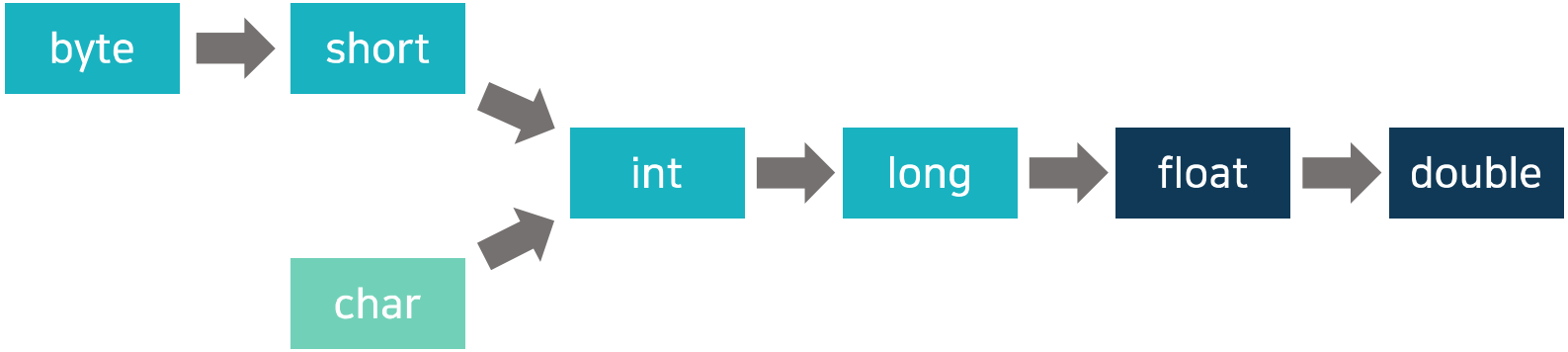
형 변환	자료형을 변경하는 것

암묵적 형 변환

작은 자료형 타입에서
큰 자료형으로의 형 변환

명시적 형 변환

큰 자료형 타입에서
작은 자료형으로의 형 변환



데이터 자료형의 변환

```
package chapter02;

public class CastingOperator {

    public static void main(String[] args) {
        char c1 = 'a';
        int n1 = 1;
        System.out.println(c1 + n1); // char타입의 변수 c1의 값이 int타입으로 변환되어 연산
        int n2 = 10;
        float n3 = 10.0f;
        System.out.println(n2 + n3); // int타입의 변수 n2의 값이 float타입으로 변환되어 연산
    }
}
```